

Agentic AI-Driven Self-Healing Cloud Infrastructure

with Explainable Anomaly Detection, Digital Twin Simulation & Autonomous Shift-Left PR Intelligence

Project Scope & Architecture Overview

An end-to-end intelligent platform that unifies proactive code quality control before deployment with autonomous, closed-loop Kubernetes self-healing during live cloud operations.

- Foundational Theory: Grounded in the SF-DTM model (Saxena & Singh, IEEE TII 2025)
- Dual-Engine Decision System: Sub-millisecond RL reflex (0.001s) paired with Gemini 2.0 Flash ReAct reasoning (2.4s)
- Safety & Explainability: KernelSHAP root-cause attribution + SimPy 0.01s pre-execution simulation dry-run gate
- Production Cloud Deployment: Microsoft Azure (Container Apps, PostgreSQL, Key Vault, Container Registry)

Abstract: Proactive & Autonomous Cloud Reliability

Project Abstract

Modern distributed cloud systems suffer from two major sources of downtime: faulty code slipping through manual reviews (Shift-Left) and slow, opaque recovery tools when live servers experience traffic spikes or crashes (Runtime Operations).

This project presents an Agentic AI-Driven Self-Healing Framework that bridges the gap between software development and live infrastructure management through a single, continuous closed loop:

- **Proactive Shift-Left Quality:** Automatically inspects Pull Requests using static AST linters, security scanners, test-coverage auditors, and Gemini LLM reasoning to catch defects before code is merged.
- **Real-Time Digital Twin Monitoring:** Continuously mirrors live microservice states and dependency call chains in an in-memory graph model (TDTdb).
- **Explainable Anomaly Detection:** Replaces black-box thresholding with Isolation Forest and KernelSHAP to show operators exactly which metrics (CPU, RAM, latency, request rate) triggered an alert.
- **Dual-Agent Consensus Decision Engine:** Evaluates incident remediation by running a fast SimiFed reinforcement learning reflex alongside deep Gemini ReAct chain-of-thought analysis.
- **Pre-Execution Safety Simulation:** Validates proposed healing actions (scale up, restart, patch limits) inside a 0.01-second SimPy discrete-event queue before touching live cloud servers.
- **Enterprise Cloud Delivery:** Deployed natively on Microsoft Azure with automated GitOps PR creation and a real-time operator control console.

Problem Statement: Why Modern Cloud Systems Fail

The Core Problem: Reactive Tools & Unchecked Code Cause Costly Downtime

Cloud applications break easily when buggy code gets merged without thorough checks, and live monitoring tools react too slowly without explaining what went wrong.

- Slow & Incomplete Code Reviews: Manual PR reviews take hours to days, and humans easily overlook hidden security flaws (hardcoded credentials, injection bugs) and missing unit tests.
- Delayed Outage Reaction: Standard cloud auto-scalers (like Kubernetes HPA) wait 3 to 5 minutes of continuous high CPU before reacting, causing service outages for users during sudden traffic surges.
- Black-Box Alert Confusion: Traditional monitoring alarms tell engineers that an error occurred, but never explain **why** it happened, forcing on-call engineers to guess root causes under high stress.
- Risky Blind Fixes: Automated recovery tools apply actions directly to live servers without testing them first, often causing destructive restart loops and cascading microservice failures.
- Disconnected Tools: Developers and operations teams use separate, disconnected tools, meaning errors caught in production are not fed back into the code review cycle to prevent repeat mistakes.

Theoretical Foundation & Literature Survey

Base Paper: SF-DTM (Saxena & Singh, IEEE TII 2025)

D. Saxena and A. K. Singh, 'A Self-Healing and Fault-Tolerant Cloud-based Digital Twin Processing Management Model,' in IEEE Transactions on Industrial Informatics, 2025.

- SimiFed Resource Estimation: Uses Cosine Similarity vector matching on past incidents to estimate needed compute without exposing raw client telemetry.
- Digital Twin State Synchronizer: Continuously tracks task execution sequences and resource allocations in a temporal database (TDTdb).
- Frequent Sequence Patterns (FSP): Mines supportive vs non-supportive task sequences to separate conflicting microservices.
- Formal Availability Metrics: Proves mathematical bounds for Mean Time Between Failures (MTBF) and Mean Time To Repair (MTTR).

Key Insights Adopted from Related Literature

Core engineering principles integrated from top IEEE & ACM publications:

- Two-Tier Cloud-Edge Collaboration (Zhang et al., IEEE IoTJ 2024): Splitting intelligence into fast local reflexes (0.001s) and deep centralized cloud reasoning.
- Multi-Expert Consensus Arbitration (JAISE 2026): Combining multiple distinct decision algorithms to prevent single-point-of-failure errors.
- Shift-Left Security & Admission Control (FGCS 2026): Enforcing security scanning, secret detection, and AST validation directly at the pull request stage.
- Idempotent Operations & Async Safety (IJPEDS 2026): Using deduplicated, non-blocking queues to prevent conflicting self-healing actions from flapping.

Novel System Architecture & Key Innovations

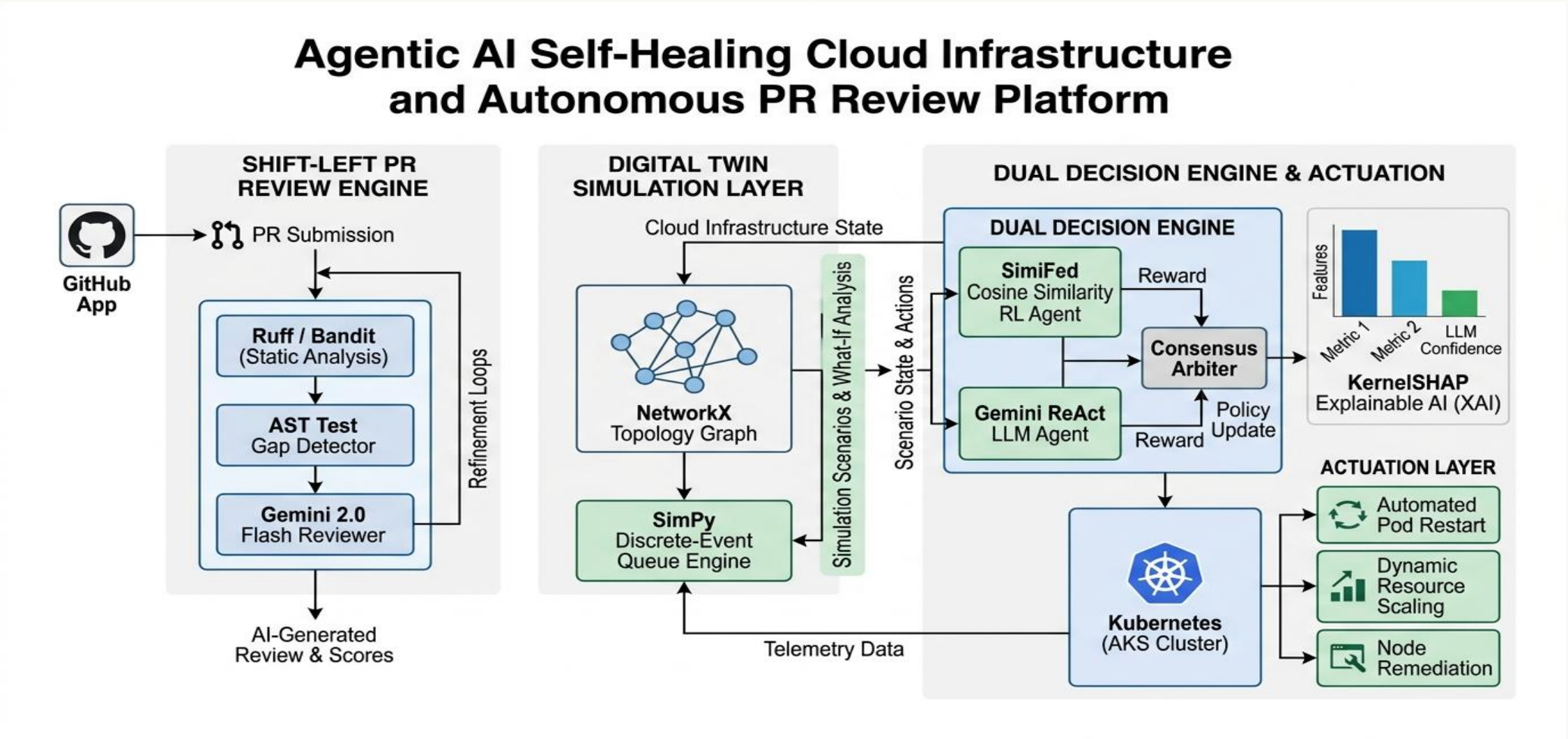
Our 5 Novel Architectural Advancements

We extended baseline literature into a complete, enterprise-grade self-healing platform with 5 major technological innovations:

- 1. Explainable AI with KernelSHAP: Replaced black-box thresholding with exact Shapley feature attributions, showing operators exactly which metrics (CPU, RAM, latency, request rate) caused the anomaly.
- 2. SimPy 0.01s Pre-Execution Simulation Gate: Added an active discrete-event M/M/c queuing simulator that tests and verifies healing actions **before** touching live cloud servers to guarantee zero downtime.
- 3. Dual Parallel Decision Engine: Engineered a dual-stream architecture pairing an instantaneous SimiFed RL reflex (0.001s) with a Gemini 2.0 Flash ReAct reasoning model (2.4s) via consensus arbitration.
- 4. Shift-Left Proactive PR Intelligence: Extended self-healing backwards to developer pull requests, automatically detecting security flaws, missing test cases, and opening self-healing GitOps PRs.
- 5. Production Cloud Deployment: Built fully automated Infrastructure as Code (IaC) on Microsoft Azure with live single-pane-of-glass operator monitoring.

End-to-End System Architecture (IEEE Block Diagram)

Agentic AI Self-Healing Cloud Infrastructure and Autonomous PR Review Platform



Methodology: Proactive Code Quality & Security Gates

How the Agent Protects Code Before Deployment (Simple Step-by-Step)

When a developer opens or updates a Pull Request on GitHub, the autonomous pipeline immediately runs the following 5-step process:

- Step 1 — Fast Static Scanning: Concurrently runs Ruff (Python linter), Bandit (security vulnerabilities like shell=True or SQL injections), Detect-Secrets (prevents API token leaks), and Pip-Audit (checks dependencies for known CVEs).
- Step 2 — AI Contextual Code Review: Injects code diffs and scanner findings into Gemini 2.0 Flash. The model reasons through logic errors, evaluates edge cases, and posts line-by-line GitHub suggestion blocks.
- Step 3 — Abstract Syntax Tree (AST) Test Gap Audit: Parses Python code trees to detect when critical public functions are modified without matching unit tests, preventing silent regression bugs.
- Step 4 — Quality Gate Enforcement: Updates the GitHub Check Run status (review-agent/quality-gate). If critical security vulnerabilities or severe flaws are detected, merging is automatically blocked.
- Step 5 — Autonomous Auto-Fix PRs: For recognized critical issues, the system can automatically create a fix branch (autoreview/fix-*), commit verified patches, and open a ready-to-merge Pull Request.

Methodology: Closed-Loop Runtime Self-Healing

How the System Heals Live Cloud Infrastructure (Simple Step-by-Step)

During live cloud operations on Microsoft Azure, the autonomous control plane monitors and repairs services through the following 5-step process:

- Step 1 — Digital Twin Telemetry Sync: Continuously pulls CPU, RAM, response latency, and request rates from Kubernetes pods and updates a live in-memory dependency graph (frontend -> checkout -> cart -> redis).
- Step 2 — Anomaly Detection: Machine learning models (Isolation Forest & XGBoost) inspect incoming telemetry vectors to catch performance degradation before full service crashes occur.
- Step 3 — Explainable Root Cause (KernelSHAP): Computes Shapley feature attribution scores to tell human operators exactly why the anomaly happened (e.g. CPU spike +0.82 vs memory leak +0.41).
- Step 4 — Dual-Agent Decision & Consensus: SimiFed RL (0.001s reflex) and Gemini ReAct LLM (2.4s reasoning) simultaneously decide the best action (SCALE_UP, RESTART_POD, PATCH_LIMITS) and reach consensus.
- Step 5 — SimPy Dry-Run Safety Gate & Actuation: Runs a 0.01-second queuing simulation to verify that the fix drops CPU load below threshold without breaking downstream services. Once verified, it executes the remediation on Azure.

5-Step Closed-Loop Operational Lifecycle

